BPF FSESSION功能详解及实践

The 4th eBPF Developer Conference (eBPFDC 2026)

董梦龙 · 天翼云科技有限公司

陕西 · 西安　　2026年4月18日

需求背景

TRACING类型的BPF程序（FENTRY、FEXIT）可以实现对内核函数的入口和出口的跟踪，是目前基于eBPF来进行内核trace的首选技术。

使用简单，可以直接进行内存（函数参数）读取；　性能高：
能够在fexit中拿到函数的参数。

```
SEC("fentry/ip_rcv")
int BPF_PROG(trace_ip_rcv_entry, struct sk_buff *skb)
{
        return 0;
}

SEC("fexit/ip_rcv")
int BPF_PROG(trace_ip_rcv_exit, struct sk_buff *skb)
{
        return 0;
}
```

```
usermode-count : 890.443 ± 0.307M/s
kernel-count   : 416.139 ± 0.055M/s
syscall-count  : 31.037 ± 0.813M/s
fentry         : 169.549 ± 0.519M/s
fexit          : 136.540 ± 0.518M/s
fmodret        : 159.248 ± 0.188M/s
rawtp          : 194.475 ± 0.144M/s
tp             : 84.505 ± 0.041M/s
kprobe         : 59.951 ± 0.071M/s
kprobe-multi   : 63.153 ± 0.177M/s
kprobe-multi-all: 4.699 ± 0.012M/s
kretprobe      : 23.740 ± 0.015M/s
kretprobe-multi: 29.301 ± 0.022M/s
kretprobe-multi-all: 3.869 ± 0.005M/s
```

局限性：

- 跟踪一个内核函数的entry和exit，要编写2个BPF程序，即使他们的逻辑相同
- 无法建立entry和exit的关联

案例1：跟踪ip_rcv函数的耗时

编写fentry：

获取当前时间戳 ->

将时间戳保存到哈希表中，key是当前进程的pid

编写fexit：

根据pid从哈希表中查找时间戳 ->

获取当前时间戳，计算耗时 ->

从哈希表中删除时间戳

问题：

- 性能低，需要进行哈希表的查找
- 难以处理复杂场景，如函数重入

功能简介

fsession：用法和kprobe.session类似，可以将单个BPF程序同时挂载到目标函数的entry和exit上。

session cookie：在entry和exit间共享的一个8字节的变量，通过bpf_session_cookie()来获取其地址。这个变量保存在栈上。

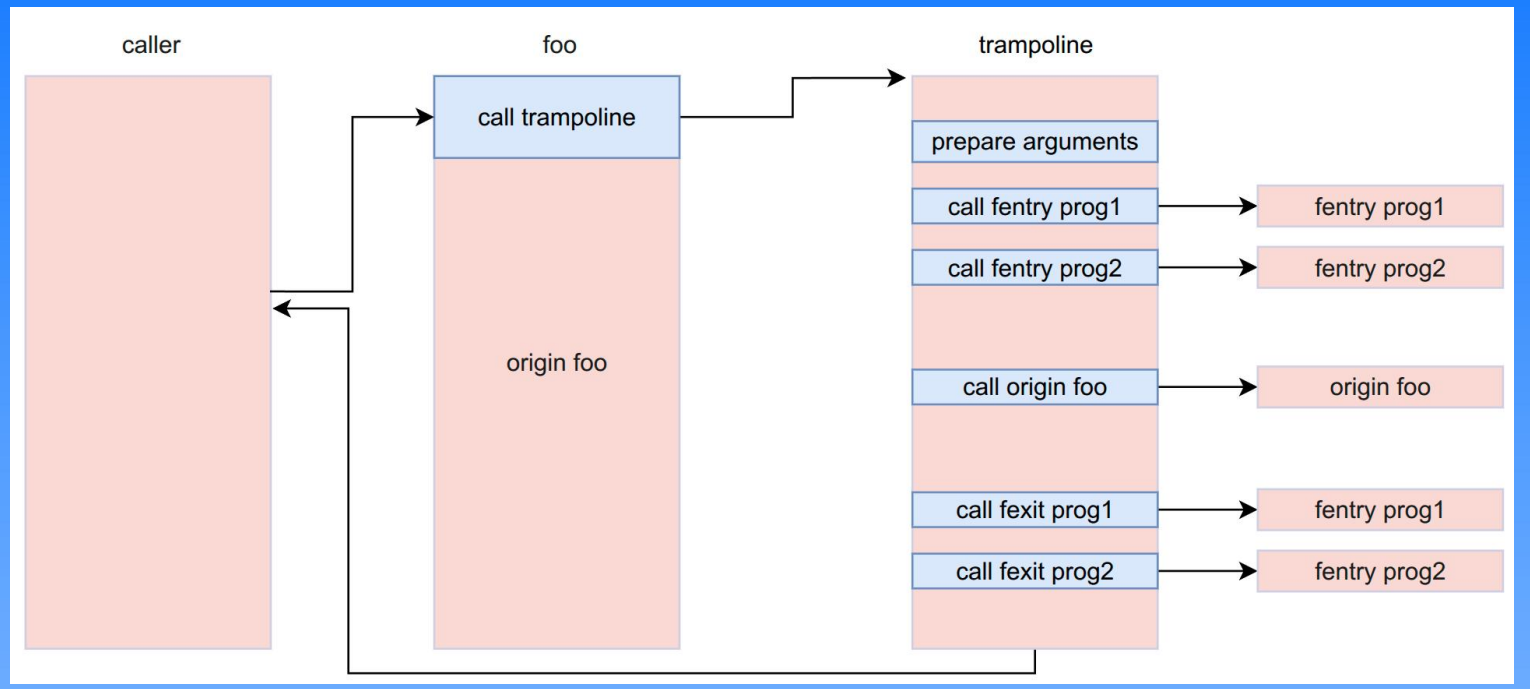
bpf_session_is_return：判断当前上下文是fentry还是fexit

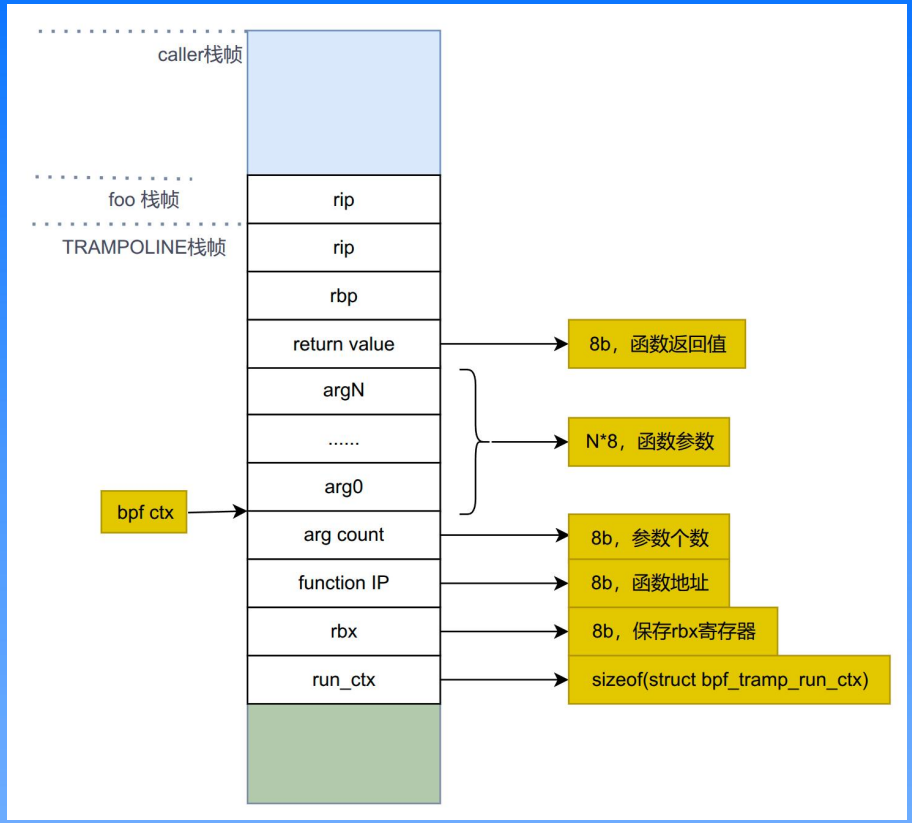
```
SEC("fsession/ip_rcv")
int BPF_PROG(trace_ip_rcv_exit, struct sk_buff *skb)
{
        if (!bpf_session_is_return(ctx)) {
                /* This is the fentry case */
                *bpf_session_cookie(ctx) = bpf_ktime_get_ns();
        } else {
                /* This is the fexit case */
                bpf_printk("elapse time: %llu\n",
                           bpf_ktime_get_ns() - *bpf_session_cookie(ctx));
        }

        return 0;
}
```

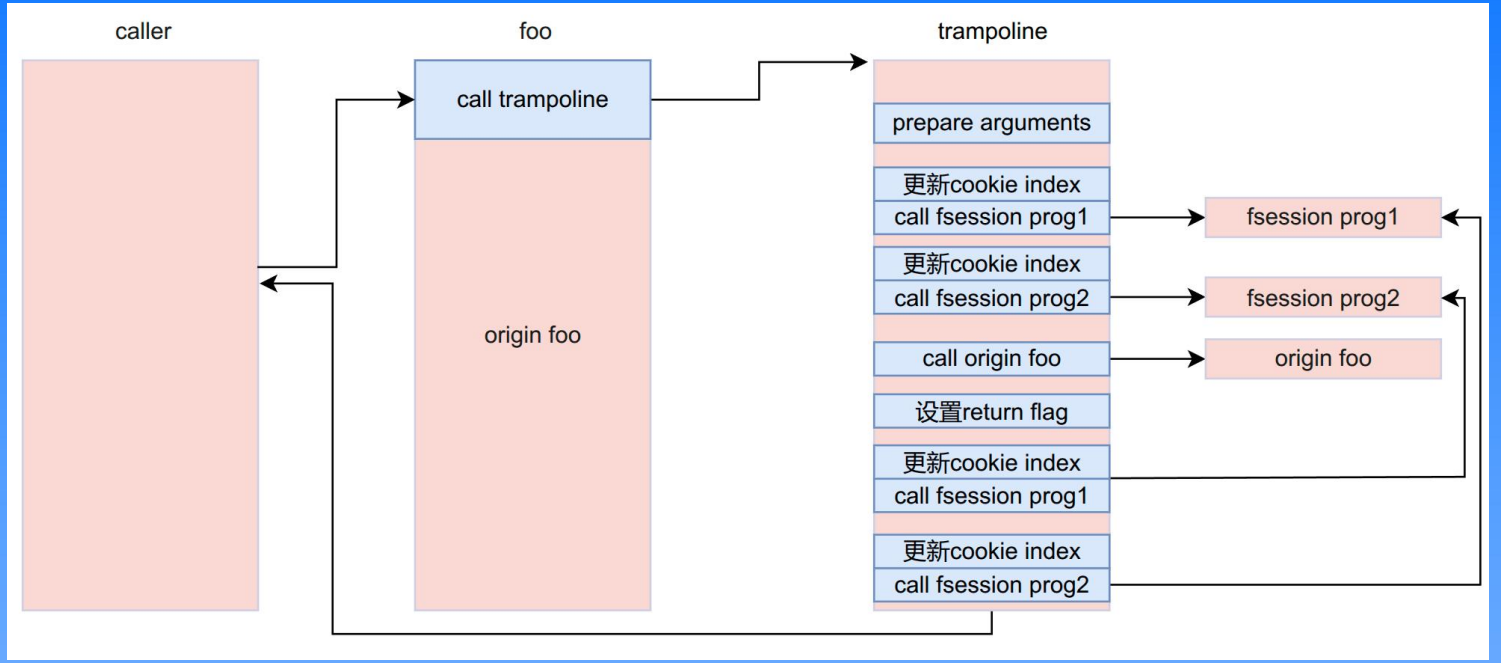
实现原理

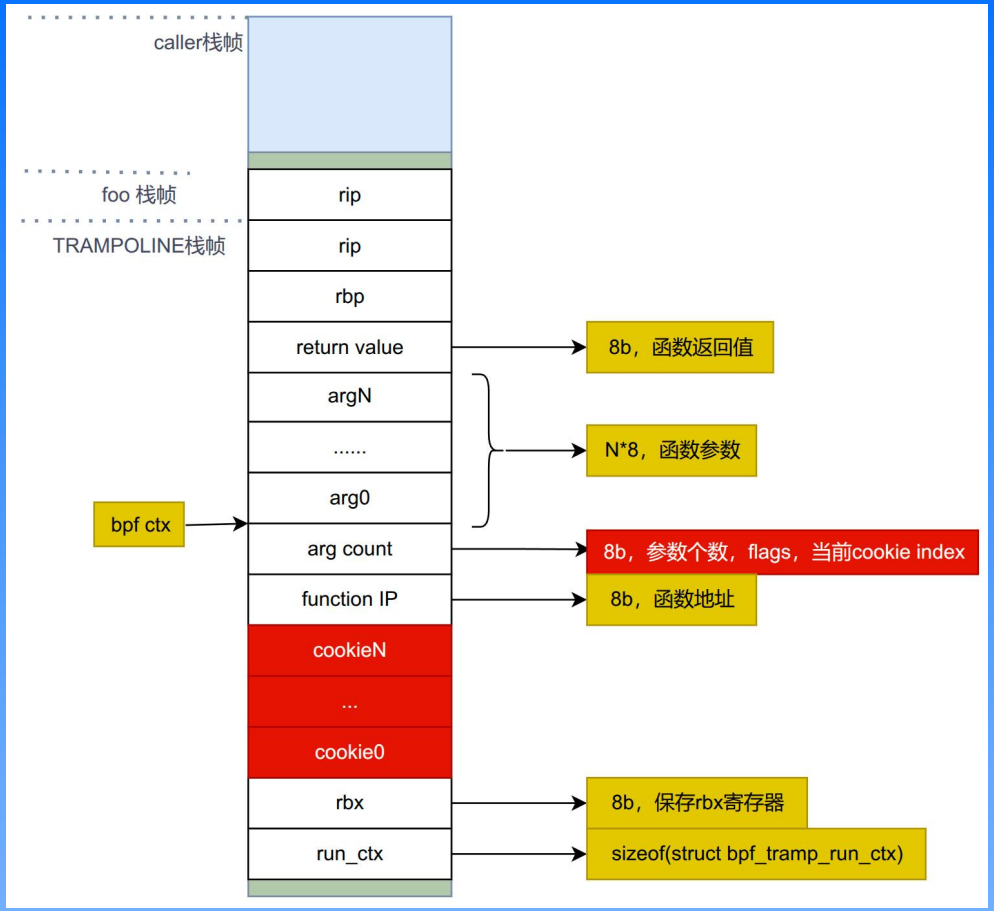
FENTRY/FEXIT基本原理：





FSESSION原理：


caller
foo
trampoline
call trampoline
origin foo
prepare arguments
更新cookie index
call fsession prog1
更新cookie index
call fsession prog2
call origin foo
设置return flag
更新cookie index
call fsession prog1
更新cookie index
call fsession prog2
fsession prog1
fsession prog2
origin foo


caller栈帧
foo 栈帧
TRAMPOLINE栈帧
rip
rip
rbp
return value
argN
.....
arg0
arg count
function IP
cookieN
...
cookie0
rbx
run_ctx
bpf ctx
8b，函数返回值
N*8，函数参数
8b，参数个数，flags，当前cookie index
8b，函数地址
8b，保存rbx寄存器
sizeof(struct bpf_tramp_run_ctx)

```
__u64 *bpf_session_cookie(void *ctx)
{
    u64 off = (((u64 *)ctx)[-1] >> BPF_TRAMP_COOKIE_INDEX_SHIFT) & 0xFF;
    return &((u64 *)ctx)[-off];
}

bool bpf_session_is_return(void *ctx)
{
    return (((u64 *)ctx)[-1] >> BPF_TRAMP_IS_RETURN_SHIFT) & 1;
}
```

应用案例

nettrace：网络报文跟踪，跟踪skb经过的内核函数。当前逻辑为：

map: m_skb_ctx, key为当前skb地址，value上有个ref（引用计数）字段

fentry:

m_skb_ctx查找skb。

找到了：value->ref++;

没找到：报文过滤 -> 构建value插入到m_skb_ctx，ref初始值为1

fexit:

m_skb_ctx查找skb，没找到就直接结束

value->ref--

如果ref为0，且value->dead，那么从m_skb_ctx中删除这个skb

skb free:

m_skb_ctx查找skb，没找到就直接结束

value->dead = true

如果ref为0，那么从m_skb_ctx中删除这个skb

```
xm@imagedong-LC1:~$ sudo nettrace --intel --intel-quiet --addr 127.0.0.1
begin trace...
**************** ffff88810b9936e8 **************
[860582.142325] [nf_hook_slow        ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S *ipv4 in chain: OUTPUT*
[860582.142345] [nft_do_chain        ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S *iptables table:nat, chain:OUTPUT* *packet is accepted*
[860582.142356] [ip_output           ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142359] [nf_hook_slow        ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S *ipv4 in chain: POST_ROUTING*
[860582.142363] [nft_do_chain        ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S *iptables table:nat, chain:POSTROU* *packet is accepted*
[860582.142375] [ip_finish_output    ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142379] [ip_finish_output2   ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142388] [__dev_queue_xmit    ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142391] [dev_hard_start_xmit ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142399] [enqueue_to_backlog  ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142415] [__netif_receive_skb_core] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142424] [ip_rcv              ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142431] [ip_rcv_core         ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142436] [nf_hook_slow        ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S *ipv4 in chain: PRE_ROUTING*
[860582.142447] [ip_local_deliver    ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142449] [nf_hook_slow        ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S *ipv4 in chain: INPUT*
[860582.142455] [ip_local_deliver_finish] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142465] [tcp_v4_rcv          ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142469] [__inet_lookup_listener] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S *tcp port is not listened*
[860582.142479] [tcp_v4_send_reset   ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S
[860582.142552] [kfree_skb           ] TCP: 127.0.0.1:40968 -> 127.0.0.1:9999 seq:762886138, ack:0, flags:S *packet is dropped by kernel*
--------------- ANALYSIS RESULT --------------------
[1] WARNING happens in __inet_lookup_listener(tcp-in):
        tcp port is not listened
    fix advice:
        check your target tcp port

[2] ERROR happens in kfree_skb(life):
        packet is dropped by kernel
    location:
        tcp_v4_rcv+0x81
    drop reason:
        NO_SOCKET

analysis finished!
```

nettrace：基于fsession的逻辑为：

map: m_skb_ctx, key为当前skb地址，value上不再有ref字段

fentry:
m_skb_ctx查找skb。
没找到：报文过滤 -> 构建value插入到m_skb_ctx,设置cookie=1

fexit:
cookie为1：处理报文

skb free:
从m_skb_ctx中删除这个skb

Thanks！